

BFS — BREADTH-FIRST SEARCH

Engineering Interview Cheatsheet for Senior Software Engineer / Technical Lead (2–3 Minute Review)

1 WHY — explore by distance layers

- Visits vertices in order of increasing distance from the source(s).
- Finds shortest paths (fewest edges) in unweighted graphs.
- Determines reachability.
- Computes levels / distances.
- Models natural “waves”: broadcasts, contagion, ripples.
- Guarantees first time we see a node is optimal (fewest edges).



Levels (distance)

- Level 0: {1}
- Level 1: {2, 3}
- Level 2: {4, 5, 6}
- Level 3: {7, 8}

2 CORE IDEA — queue frontier

- Maintain a FIFO queue of the current frontier.
- Dequeue a node, discover all unvisited neighbors.
- Mark visited and enqueue neighbors.
- Process level by level (all nodes at distance d before $d+1$).

Graph	Step	Dequeue	Queue (front → back)	Visited	Level
	0	–	[1]	{1}	0
	1	1	[2, 3]	{1, 2, 3}	1
	2	2	[3, 4, 5]	{1, 2, 3, 4, 5}	2
	3	3	[4, 5, 6]	{1, 2, 3, 4, 5, 6}	2
	4	4	[5, 6]	(no change)	2
	5	5	[6, 7, 8]	{..., 7, 8}	3
	6	6	[7, 8]	(no change)	3
	7	7	[8]	(no change)	3
	8	8	[]	(no change)	3

Discovery order: 1, 2, 3, 4, 5, 6, 7, 8

3 INTERNALS

- **Graph representation:** adjacency list preferred.
- **Visited on ENQUEUE** (not dequeue) to avoid duplicates in queue.
- **Track metadata:**
 - $\text{dist}[v]$ = distance (level) from source (init ∞)
 - $\text{parent}[v]$ = predecessor for path reconstruction
- **Multi-source BFS:** enqueue all sources ($\text{dist} = 0$).
- **Disconnected graphs:** run BFS from every unvisited vertex.
- **Implicit state graphs:** generate neighbors on-the-fly (e.g., puzzles, configurations).
- **Memory:** dominated by frontier width (max nodes in queue) and visited set.
- **Bidirectional BFS:** run BFS from source and goal simultaneously; stop when frontiers meet.

4 COMPLEXITY

Scenario	Time Complexity	Space Complexity
Adjacency List	$O(V + E)$	$O(V)$
Adjacency Matrix	$O(V^2)$	$O(V)$
Tree (n nodes)	$O(n)$	$O(n)$
Grid ($R \times C$)	$O(R \times C)$	$O(R \times C)$

- Each vertex enqueued at most once.
- Each edge inspected at most once (undirected: twice).
- V = vertices (states), E = edges (transitions).
- Space is $O(V)$ for visited + queue; in practice $\approx$ frontier width + visited.

5 TRADE-OFFS

Strengths

- ✓ Guarantees shortest path in unweighted graphs.
- ✓ Simple, iterative, easy to implement and reason about.
- ✓ Finds all nodes within radius k naturally (levels).
- ✓ Works with multiple sources.

Limitations

- ✗ High memory when frontier is wide (b^d in branching).
- ✗ Not for weighted shortest paths.
- ✗ Explores many nodes even if goal is near but behind many alternatives (exhaustive by radius).

When NOT to use BFS

- Deep, narrow solutions → DFS / Iterative Deepening.
- Weighted positive edges → Dijkstra's Algorithm.
- Negative edges → Bellman-Ford (or SPFA).

6 DISCUSSION — common talking points

1. Why does BFS return shortest path in unweighted graphs?
2. When should we mark visited: enqueue or dequeue? Why?
3. How does level-order processing emerge from the queue?
4. How do you reconstruct the path from source to target?
5. How does multi-source BFS change the solution space?
6. How does bidirectional BFS reduce work?
7. How would you handle 0-1 edge weights (0-1 BFS)?
8. What if the graph is infinite or implicit?
9. How would you avoid revisiting states / handle cycles?
10. How would you parallelize or distribute BFS?

7 RELATED ALGORITHMS & CONCEPTS

	Queues	FIFO data structure that drives BFS.
	DFS	Explores depth-first; uses stack/recursion.
	Dijkstra	Shortest paths in graphs with non-negative weights.
	0-1 BFS	Special case for edge weights {0, 1} using deque.
	Topological Traversal	BFS on DAG (Kahn's algorithm) for topo sort.
	Connected Components	Run BFS from each unvisited vertex.
	Bipartite Check	Color with 2 colors via BFS; detect odd cycles.
	Flood Fill	BFS on grids for area fill / region detection.

8 PRODUCTION USE CASES

	Network discovery	Find all reachable hosts / hops (traceroute-like).
	Web crawling	Level-order crawl, respect depth limits.
	Social graph degrees	Friends-of-friends, k -degree connections.
	Service dependencies	Blast radius / impact analysis across services.
	Shortest transformations	Word ladder, gene mutation, state transitions.
	Grid-based routing	Shortest path in mazes, maps, games.
	Broadcast propagation	Estimate spread in networks / time-to-live.
	Recommendation neighborhoods	k -hop neighborhoods, similarity expansion.
	Garbage collector variants	Traverse object graphs (stop-the-world, incremental).

9 INTERVIEW THINKING CHECKLIST

1. Define the state & edges (what is a node? how to move?).
2. Is the graph unweighted? If not, choose the right algorithm.
3. Identify source(s) and goal(s); is it reachability or distance?
4. Choose BFS: plan queue + visited; decide enqueue timing.
5. Decide what metadata to store: dist, parent, level, etc.
6. Define termination: found goal? explored all? depth limit?
7. Analyze complexity in terms of V and E (or branching b , depth d).
8. Consider disconnected graphs, duplicates, self-loops.
9. Consider edge cases: empty graph, single node, cycles.
10. Consider memory: frontier width, visited representation.

10 SENIOR NOTES — deeper insights

- ★ Visited-on-enqueue prevents queue explosion and redundant work.
- ★ Frontier width (max queue size) determines peak memory; in worst case can be $\Theta(V)$.
- ★ Multi-source BFS models “nearest” facility / origin problems elegantly.
- ★ Bidirectional BFS reduces effective depth from d to $\sim d/2 \rightarrow$ huge savings.
- ★ Implicit graphs: generate neighbors lazily; ensure bounded state space and dedup (visited hashing / canonicalization).
- ★ Shortest means fewest edges in unweighted graphs; if all edges have equal weight w , BFS gives min total weight $= w \times (\#edges)$.
- ★ BFS is naturally iterative and easy to parallelize at each frontier (level).
- ★ Beware of heavy neighbor generation cost; cache or short-circuit filters.

11 MENTAL HOOK

BFS expands a wavefront, so the first arrival uses the fewest edges.

DFS — DEPTH-FIRST SEARCH

Engineering Map for Senior Software Engineer / Technical Lead Interview Prep (2–3 minute review)

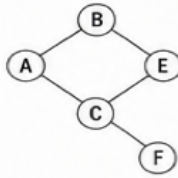
1 WHY

- Explores a graph by going as deep as possible along each branch before backtracking.
- Reveals structural properties of graphs.
- Finds connected components (undirected).
- Detects cycles (directed & undirected).
- Produces finish times useful for topological ordering (DAGs).
- Foundation for SCCs, articulation points, bridges, and many analyses.
- Natural for backtracking / search problems (maze, N-puzzle, constraint search).
- Simple, fast: linear time on adjacency lists.

2 CORE IDEA

Visit a node, then recursively (or via stack) explore an unvisited neighbor. When none remain, backtrack.

Example graph (start at A)



Discovery order: A, B, D, E, C, F

Finish order: D, E, B, F, C, A

(reverse finish order: A, C, F, B, E, D)

Step	Action	Node	Disc	Finish
1	discover	A	1	—
2	discover	B	2	—
3	discover	D	3	—
4	finish	D	3	4
5	discover	E	5	—
6	finish	E	5	6
7	finish	B	2	7
8	discover	C	8	—
9	discover	F	9	—
10	finish	F	9	10
11	finish	C	8	11
12	finish	A	1	12

3 INTERNALS

- Two implementations:
 - Recursive call stack (implicit).
 - Explicit stack (iterative).
- Each vertex has:
 - color: WHITE (unseen), GRAY (discovered, in current DFS path), BLACK (finished).
 - parent[u]: predecessor in DFS tree/forest.
 - disc[u]: discovery timestamp (pre-order).
 - finish[u]: finish timestamp (post-order).
- Rules:
 - Mark GRAY and set disc[u] before exploring neighbors.
 - After all neighbors, set BLACK and finish[u].
- DFS repeats from every WHITE vertex to cover disconnected graphs → “forest” of DFS trees.
- Stack depth = recursion depth. Worst case $O(V)$ (e.g., path graph).
- For faithful iterative DFS order, store iterator/index of next neighbor to process with each stack frame.

Colors	
○	WHITE unseen
●	GRAY active
●	BLACK finished

4 COMPLEXITY

Representation	Time	Why
Adjacency List	$O(V + E)$	Each vertex and edge considered $O(1)$ times.
Adjacency Matrix	$O(V^2)$	For each vertex, we scan all possible neighbors.

Space (auxiliary):

- Visited/color, parent, disc, finish: $O(V)$
- Call stack (or explicit stack): $O(\text{depth}) \subseteq O(V)$
- Total space: $O(V)$

Typical Depth / Shape

Graph Shape	Depth	Notes
Balanced tree ($b=k$)	$O(\log_k V)$	Shallow.
Skewed tree / path	$O(V)$	Worst case depth.
Grid ($m \times n$)	$O(mn)$	Can be deep.

Work Accounting (Adjacency List)

- Each vertex is discovered and finished once.
- Each edge (u, v):
 - Undirected: examined twice ($u \rightarrow v$ and $v \rightarrow u$).
 - Directed: examined once ($u \rightarrow v$).
- Therefore total time is $O(V + E)$.

5 TRADE-OFFS

✓ Strengths

- Low memory for frontier (keeps one path; stack $\leq$ depth).
- Natural for deep structure discovery and backtracking.
- Simple and fast (linear time on adjacency lists).
- Finish times unlock many algorithms (topological sort, SCC).
- Works on disconnected graphs (forest).

✗ Limitations

- Does NOT guarantee shortest path in unweighted graphs (BFS does).
- May go very deep → stack overflow in recursion.
- Traversal order depends on neighbor iteration order.
- On huge or adversarial depth, recursion is risky.

🚫 When NOT to use DFS

- Nearest / fewest edges path → use BFS.
- Shortest path with weights → Dijkstra / Bellman-Ford.
- Extremely deep graphs where recursion can't be avoided and explicit stack not feasible.
- When you need level-by-level guarantees.

6 DISCUSSION (Interview Prompts)

1. Recursive or iterative DFS? Trade-offs? When choose which?
2. How do you detect a cycle in:
 - Undirected graphs? (neighbor seen with parent[u] $\neq$ v)
 - Directed graphs? (back-edge to GRAY ancestor)
3. How does DFS give topological order in DAGs? (Sort vertices by decreasing finish time.)
4. What do discovery/finish timestamps enable? (Edge classification, SCC, intervals/nesting.)
5. How do you find connected components? (Run DFS from each WHITE vertex; count trees.)
6. How do you find Strongly Connected Components? (Kosaraju / Tarjan using DFS and finish times.)
7. How do you reconstruct a path found by DFS? (Follow parent[] from target to source.)
8. How do you classify edges using timestamps/colors? (Tree, Back, Forward, Cross.)
9. How would you safeguard against deep recursion? (Iterative stack, depth cap, or tail limits.)
10. What's the impact of input order on DFS? On correctness?

7 RELATED

- **BFS (Breadth-First Search)** — shortest path in unweighted graphs, level order.
- **Recursion & Stacks** — DFS is a canonical use of the call stack.
- **Topological Sort** — order vertices by decreasing finish time in DAGs.
- **Strongly Connected Components (SCC)** — Kosaraju / Tarjan.
- **Articulation Points & Bridges** — find cut vertices/edges via DFS low-link values.
- **Backtracking & Constraint Search** — explore, prune, backtrack patterns.
- **Flood Fill / Region Fill** — grid DFS to color or count regions.
- **Tree Traversals** — DFS on trees: pre/in/post-order traversals.

8 PRODUCTION USES

- **Filesystem / AST traversal** — enumerate files, directories, or syntax trees.
- **Dependency analysis** — build order, impact analysis, cascade detection.
- **Dead code elimination** — reachability from program entry points.
- **Cycle detection** — configuration, build graphs, package managers.
- **Graph validation** — detect invalid cycles or disconnected required pieces.
- **Maze / state-space exploration** — game AI, planning, symbolic execution (with pruning).
- **Garbage collection (mark phase)** — DFS mark from roots.
- **Web crawling / link analysis** — depth-limited crawl strategies.
- **Infrastructure dependency audits** — microservices, cloud resources, IaC graphs.

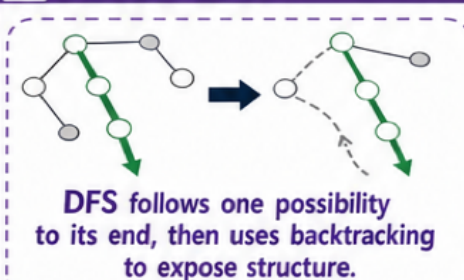
9 INTERVIEW THINKING ROADMAP

1. Define graph semantics. Directed or undirected? What is a vertex? What are neighbors? Any weights?
2. Choose state representation. visited/color, parent, timestamps.
3. Pick implementation. Recursive for clarity, or explicit stack for control/safety.
4. Define pre-action (upon discover) and post-action (upon finish). Set disc[] before exploring; set finish[] after.
5. Cycle detection rules. Undirected (parent check) vs directed (back-edge to GRAY).
6. Handle disconnected graphs. Outer loop over all vertices; start DFS at each WHITE.
7. Depth control / safety. Anticipate max depth; iterative or cap recursion if necessary.
8. Complexity. State time in terms of V and E ; space considerations.
9. Test & validate. Empty, single vertex, line, star, cycles (dir/undir), disconnected, dense graphs, deep chains.

10 SENIOR NOTES

- Recursive DFS is concise but hides the real resource: call stack. Know your platform limits.
- Iterative DFS must preserve neighbor iterator state per frame to match recursive order exactly.
- Directed vs undirected cycle tests are different — get parent/back-edge rules right.
- Finish times enable powerful decompositions:
 - Topological order, SCCs, condensation DAGs.
 - Edge classification and interval nesting.
- Input adjacency order affects traversal order (and thus which valid DFS tree you get) but not reachability or correctness.
- For adversarial inputs (long chains), cap recursion depth or switch to iterative to ensure stability.

11 MENTAL HOOK



DFS QUICK REFERENCE

Recursive Pseudocode	Iterative (Explicit Stack) Sketch
<pre>time = 0 def dfs(u, parent): global time color[u] = GRAY disc[u] = time = time + 1 for v in adj[u]: if color[v] == WHITE: parent[v] = u dfs(v, u) color[u] = BLACK finish[u] = time = time + 1</pre>	<pre>stack = [(u, None, iter(adj[u]))] color[u] = GRAY parent[u] = None disc[u] = 0 while stack: u, p, it = stack[-1] try: v = next(it) if color[v] == WHITE: parent[v] = u color[v] = GRAY disc[v] = ++time stack.append((v, u, iter(adj[v]))) except StopIteration: color[u] = BLACK finish[u] = ++time stack.pop()</pre>
Edge Classification (Directed Graph)	
Tree Edge	(u, v) where v is discovered by exploring u.
Back Edge	(u, v) where v is GRAY (ancestor of u).
Forward Edge	(u, v) where v is BLACK and disc[u] < disc[v] and u is ancestor of v.
Cross Edge	(u, v) where v is BLACK and u is not ancestor of v.
Invariant: GRAY vertices are exactly those on the current DFS path (recursion stack).	

TOPOLOGICAL SORT

Engineering Map for Senior Software Engineer / Technical Lead Interview Prep (2–3 Minute Review)

1. WHY

- Linearize dependencies so that every prerequisite appears before its dependents.



Scheduling



Build / Package Order



Order Validation



Workflow Orchestration



DB Migration Ordering



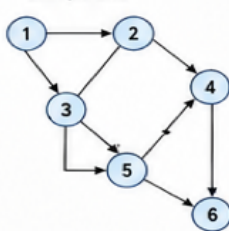
Compiler / Pass Pipelines

At its heart: respect constraints, produce a sequence that is executable.

2. CORE IDEA

- Valid only for DAG (Directed Acyclic Graph).
- For every edge $u \rightarrow v$, u appears before v .
- Multiple valid orders may exist.

Example DAG



Valid Orders (examples)

```
1 2 3 4 5 6
1 2 3 5 4 6
1 3 2 4 5 6
1 3 2 5 4 6
1 3 5 2 4 6
...
```

Not unique unless the DAG is a total order.

3. INTERNALS

Two classic approaches: Kahn's (BFS) and DFS.

KAHN'S ALGORITHM (BFS)

- Compute $\text{indegree}[v]$ for all v .
- Enqueue all nodes with indegree 0.
- Repeatedly dequeue u , emit it, decrement indegree of neighbors.
- If emitted count $< |V| \rightarrow$ cycle exists.

Indegree (example)

```
1: 0
2: 1 (from 1)
3: 1 (from 1)
4: 2 (from 2,3)
5: 1 (from 3)
6: 2 (from 4,5)
```

DFS-BASED (POSTORDER)

- Use colors: WHITE (unvisited), GRAY (visiting), BLACK (done).
- DFS(u): mark GRAY, visit neighbors. On finish, append u to list, mark BLACK.
- If edge to GRAY node $\rightarrow$ cycle.
- Reverse finishing order for topological order.

Color Legend

WHITE ○
GRAY ●
BLACK ●

★ Deterministic Order

Use min-heap / priority queue for Kahn or visit neighbors in sorted order for DFS.

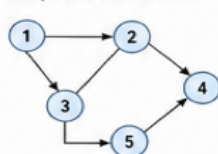
4. COMPLEXITY

Approach	Graph Representation	Time	Auxiliary Space (extra)
Kahn (BFS)	Adjacency List	$O(V + E)$	$O(V)$ (indegree + queue)
DFS (Postorder)	Adjacency List	$O(V + E)$	$O(V)$ (recursion stack + color + output)
Kahn / DFS (Deterministic)	Adjacency List (min-heap / sorted)	$O(V + E \log V)$	$O(V)$
Kahn / DFS	Adjacency Matrix	$O(V^2)$	$O(V)$ (indegree)

Indegree storage: $O(V)$. Output order: $O(V)$. Graph: $O(V + E)$ (idx).

KAHN TRACE (INDEGREE + QUEUE)

Example DAG (same as above)

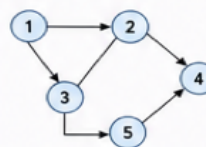


Step	Queue (front $\rightarrow$ back)	Pop	Emit Order	Decrement Neighbors (and new indegrees)	Indegrees [1 2 3 4 5 6]
0	[1]	—	—	—	0 1 1 2 2
1	[2, 3]	1	1	2 $\rightarrow$ 0, 3 $\rightarrow$ 0	0 0 0 1 2
2	[3, 2]	2	1 2	4 $\rightarrow$ 1	0 0 0 1 2
3	[3, 4]	3	1 2 3	4 $\rightarrow$ 0, 5 $\rightarrow$ 0	0 0 0 0 2
4	[4, 5]	4	1 2 3 4	6 $\rightarrow$ 1	0 0 0 0 1
5	[5]	5	1 2 3 4 5	6 $\rightarrow$ 0	0 0 0 0 0
6	[6]	6	1 2 3 4 5 6	—	0 0 0 0 0

All 6 nodes emitted $\rightarrow$ DAG. Emitted order: 1 2 3 4 5 6

DFS TRACE (FINISH ORDER)

DFS Order (assume visit neighbors in ascending order)



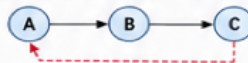
Finish Times (postorder)

Node	Finish Time
6	1
5	2
4	3
2	4
3	5
1	6

Reverse finish order $\rightarrow$ Topological Order

1 3 2 4 5 6

Cycle Detection (DFS): encountering a GRAY node via $u \rightarrow v$ means a back-edge $\rightarrow$ cycle.



WHITE: unvisited
GRAY: visiting (in stack)
BLACK: done

5. TRADE-OFFS

KAHN (BFS)

- ✓ Explicit notion of readiness (indegree 0).
- ✓ Naturally exposes parallel "batches".
- ✓ Easy to count/report all cycles (by leftovers).
- Needs queue (can be large: $\Theta(V)$ in worst case).

DFS (POSTORDER)

- ✓ Compact memory, often simpler code.
- ✓ Natural finish order; good for many analyses.
- Recursion depth up to $\Theta(V)$ (risk of stack overflow).
- Parallelism/scheduling not as direct.

When NOT to use topological sort alone

- Graph has cycles $\rightarrow$ need SCC/feedback handling.
- Weights / durations / resources $\rightarrow$ use scheduling algorithms (critical path, earliest start, etc.).

6. DISCUSSION PROMPTS

- 🔍 How do you detect a cycle? What result do you return?
- ❓ When is the order unique? How to check?
- 🔍 How to get the lexicographically smallest order?
- 🔍 How to enumerate all topological orders?
- ⚙️ How to get parallel execution levels (batches)?
- 🔍 Why does DFS reverse finish order produce a valid topo order?
- 🔍 How to handle dynamic edge insertions/deletions?
- ➡️ What if edge direction is ambiguous / reversed?
- 🏠 What about disconnected DAG (multiple components)?

7. RELATED TOPICS

DAG	Directed Acyclic Graph.
DFS	Depth-First Search.
BFS / Kahn	Breadth-First (indegree) approach.
Cycle Detection	Back-edge (DFS) or leftover nodes (Kahn).
SCC	Strongly Connected Components (Tarjan/Kosaraju).
Critical Path	Longest path in DAG (durations/dependencies).
Build Systems	Make, Bazel, Gradle.
Dependency Injection	Object graph instantiation order.
Partial Orders	Theoretical foundation (poset).

8. PRODUCTION USE CASES

- 🔧 Build / Package Managers (e.g., npm, Maven, Bazel)
- 📅 Course Prerequisites Planning
- 🔄 Workflow / Pipeline Orchestration (Airflow, Argo)
- 🗄️ Database Migration Ordering
- 📊 Spreadsheet Recalculation (cell dependencies)
- 🔧 Compiler Pass Scheduling
- 🌐 Infrastructure as Code Resource Graphs (Terraform, CloudFormation)
- 📊 ETL / Data Pipeline Dependencies
- 🏠 Microservices Startup / Shutdown Ordering

9. INTERVIEW THINKING FRAMEWORK

- 1 Define the edge direction ($u \rightarrow v$ means u before v).
- 2 Verify DAG requirement; understand failure mode (cycle).
- 3 Choose approach: Kahn (BFS) vs DFS (postorder).
- 4 Maintain metadata: indegree (Kahn) or color & stack (DFS).
- 5 Decide deterministic tie policy (heap/sorted neighbors).
- 6 Define result contract: order or cycle diagnostics.
- 7 Analyze complexity in terms of V and E .
- 8 Consider disconnected components & duplicate edges.
- 9 Test: empty graph, single node, chain, star, cycle, large wide/long.

10. SENIOR NOTES

- ★ Topological order is not unique by default.
- ★ Edge direction confusion is a frequent source of bugs.
- ★ Zero-indegree batches expose potential parallelism but not resource feasibility.
- ★ Cycles should return actionable diagnostics (e.g., cycle path).
- ★ Stable output (deterministic tie-breaking) aids caching, reproducibility, and diffs.
- ★ Dynamic DAG maintenance (online updates) is a different problem space (incr. algorithms).

11. MENTAL HOOK

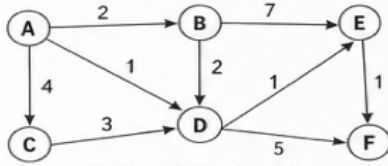
Topological sort turns a partial order into one executable sequence—only when no cycle argues back.

SHORTEST PATH

Senior Software Engineer / Technical Lead Interview Prep Cheat Sheet (2-3 Minute Review)

1 WHY

- **Goal:** minimize cost/distance/time/hops between nodes in a graph.
- **Enables:** routing, navigation, planning, scheduling, dependency resolution, latency/cost analysis.
- **Graph models:** road networks, packet networks, job DAGs, call graphs, state spaces, etc.



Weighted Graph (directed)

Example: shortest path $A \rightarrow F$ is $A-D-E-F$ with cost $1+1+1=3$.

2 CORE IDEA

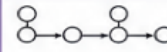
- Path cost = sum of edge weights along the path.
- **Relaxation:** for edge (u, v) with weight $w(u, v)$ if $\text{dist}[u] + w(u, v) < \text{dist}[v]$ then $\text{dist}[v] = \text{dist}[u] + w(u, v)$ and $\text{parent}[v] = u$.
- $\text{parent}[]$ lets us reconstruct the path.

Relaxation ($u \rightarrow v, w$)
if $\text{dist}[u] + w < \text{dist}[v]$:
 $\text{dist}[v] = \text{dist}[u] + w$
 $\text{parent}[v] = u$

Problem Variants

- **Single-Source Shortest Path (SSSP):** from one source.
- **Single-Pair Shortest Path:** between given s and t .
- **All-Pairs Shortest Path (APSP):** between every pair.

3 INTERNALS (ALGORITHMS AT A GLANCE)



BFS (Unweighted)
All edges weight = 1. Layered traversal. Computes min hops.



Dijkstra (Nonnegative Weights)
Greedy by smallest dist using min-heap. Finalizes nodes. Requires $w \geq 0$.



Bellman-Ford (Negative Weights)
Relax all edges $V-1$ times. Detects negative cycles reachable from source.



DAG Shortest Path
Topologically sort DAG, relax edges once in order. Works with negative weights.



Floyd-Warshall (All-Pairs)
DP over intermediate nodes k . Handles negative edges (no neg cycles). $O(V^3)$ time, $O(V^2)$ space.



A* Search (Heuristic)
Best-first on $f(n) = g(n) + h(n)$. h = admissible (never overestimates) and consistent $\rightarrow$ optimal & efficient.

4 COMPLEXITY COMPARISON

Algorithm	Use Case	Time Complexity	Space	Edge Weights	Key Constraints / Notes
BFS (Unweighted)	SSSP	$O(V + E)$	$O(V)$	All = 1	Min hops. Use queue.
Dijkstra (Min-Heap)	SSSP	$O((V + E) \log V)$	$O(V)$	$w \geq 0$	Greedy. Use min-heap. Stale entries OK.
Bellman-Ford	SSSP	$O(VE)$	$O(V)$	Any (neg ok)	Detects negative cycle. Slower.
DAG Shortest Path	SSSP on DAG	$O(V + E)$	$O(V)$	Any	Topo order required.
Floyd-Warshall (APSP)	APSP	$O(V^3)$	$O(V^2)$	Any (neg ok)	Detect neg cycles via $\text{dist}[i][i] < 0$.
A* Search	SSSP (goal t)	Worst-case Exponential-like ($= O(b^d)$)	$O(V)$	$w \geq 0$	Goal-directed. Depends on heuristic quality.

V = number of vertices, E = number of edges, b = branching factor, d = depth to goal.

Practical Guide

- Sparse graphs: Dijkstra ($w \geq 0$) or Bellman-Ford (neg edges).
- Dense graphs / many queries (APSP): Floyd-Warshall or Johnson.
- DAG with many queries: DAG shortest path (per source) or DP.
- Good heuristic available: A*.

5 TRADE-OFFS



- Algorithm choice depends on: edge weights (sign), query pattern (SSSP vs APSP), graph density, heuristic.



- **Trade-offs:** correctness guarantees vs speed, memory, and preprocessing cost.



- **When NOT to use shortest-path:**
 - MST (Minimum Spanning Tree) solves different objective (connect all nodes with min total cost, not paths).



- Negative cycles reachable from source $\Rightarrow$ no finite shortest path.



- Dynamic graphs (frequent updates) may need recomputation or incremental algorithms.

6 DISCUSSION POINTS

- Why does Dijkstra fail with negative edges? Provide counterexample.
- What are "stale" entries in the heap? How are they handled?
- Why don't we need decrease-key with a binary heap? (Insert duplicates & skip stale.)
- How do you reconstruct the actual path?
- How do you handle overflow and "infinity"?
- How do you detect and report negative cycles?
- How would you make A* optimal? (Admissible & consistent heuristic.)
- When is bidirectional search helpful?
- How do you handle multiple sources efficiently?
- What changes if edge weights are floating point?

7 RELATED CONCEPTS



- **BFS & Layered Traversal** (Unweighted graphs)



- **Priority Queues / Heaps** (Min-heap, pairing heap)



- **Dynamic Programming / Relaxation Principle**



- **Topological Sort** (for DAG shortest path)



- **Heuristics & A*** (Informed search)



- **MST vs Shortest Path** (Different objectives)



- **Johnson's Algorithm** (APSP with sparse & neg edges)



- **Routing Protocols:** Dijkstra in OSPF, IS-IS, link-state

8 PRODUCTION USE CASES



- **Road navigation** (GPS, traffic-aware routing)



- **Network routing** (OSPF / link-state protocols)



- **Logistics & delivery** (warehouse to customer)



- **Build & task dependency** (critical path / latency)



- **Game AI** (NPC pathfinding)



- **Cloud & data center topology design**



- **Fraud detection** (min-cost / min-risk links)



- **Database query planning** (join order / costs)



- **Robotics & motion planning** (cost maps)

9 INTERVIEW THINKING FLOW

- 1 Define graph semantics (directed? weighted? static?).
- 2 Check edge weights (unweighted / nonneg / negative?).
- 3 Understand query pattern (SSSP / single-pair / APSP?).
- 4 Choose algorithm that fits constraints.
- 5 Initialize $\text{dist}[] (\infty)$, $\text{parent}[] (-1)$; $\text{dist}[\text{source}] = 0$.
- 6 Apply relaxation invariant until no improvement.
- 7 Termination condition (queue empty / heap empty / $V-1$ passes / topo order done / goal reached).
- 8 Reconstruct path from $\text{parent}[]$ (handle unreachable).
- 9 Handle edge cases: overflow, disconnected, negative cycle, multiple paths, tie-breaking.
- 10 State complexity in terms of V and E .

10 SENIOR NOTES



"Shortest" is only as good as the metric. Define it explicitly (time, cost, hops, risk, energy).



Dijkstra requires nonnegative weights. Ensure data contracts and ETL pipelines guarantee it.



Floating weights and ties affect determinism. Define tie-breaking (e.g., lexicographic path, node id).



Graph snapshots matter. Use consistent views for correct results.



Heuristic correctness first, speed second. Validate admissibility & consistency.



Path output cost is real: storing parents/paths can be $O(V)$ space. Watch memory at scale.

11 MENTAL HOOK



Shortest-path algorithms differ mainly in which relaxation is safe to trust next.



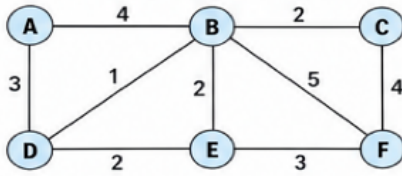
MINIMUM SPANNING TREE

Senior Software Engineer / Technical Lead Interview Prep – 2–3 Minute Review

1 WHY

- Connect all vertices with minimum total edge cost and no cycles.
- Used for network design, clustering backbone, infrastructure planning, cost optimization.

Example Weighted Graph



An MST for this graph has total cost 10 (e.g., edges: BD(1), BC(2), DE(2), EF(3), AD(3)).

4 COMPLEXITY

- Kruskal:** sort edges + Union-Find
 $O(E \log E) \approx O(E \log V)$; DSU ops $\sim O(\alpha(V))$ (inverse Ackermann, effectively constant).
- Prim with binary heap:** $O(E \log V)$
- Prim with adjacency matrix:** $O(V^2)$
- Space:** $O(V + E)$

Algorithm Selection Guide

Graph Type	Edge Density	Preferred	Why
Sparse	$E \ll V^2$ ($\approx V \log V$)	Kruskal or Prim (heap)	Lower time complexity
Dense	$E \approx V^2$	Prim (matrix)	$O(V^2)$ preferable to $O(E \log V)$
Very Large, External	Huge E	Kruskal (external sort)	Sorting can be external/parallel

V = number of vertices, E = number of edges
 $\alpha(V)$ = inverse Ackermann function (< 5 for any practical V)

7 RELATED

	Union-Find (Disjoint Set) Cycle detection, Kruskal core.
	Heaps (Priority Queue) Prim's algorithm (min-heap).
	Greedy Algorithms Make locally optimal, provably safe choices.
	Cuts & Cycles Cut property (add), Cycle property (discard).
	Clustering MST underlies single-link hierarchical clustering.
	Shortest Path (Dijkstra) Different objective: path vs global connection.
	Steiner Tree Connect subset with optional Steiner nodes.

10 SENIOR NOTES

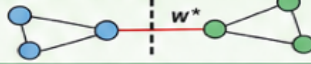
- Cost-Minimal but Fragile:** MST minimizes cost, not reliability. Real networks need redundancy (add edges, k -edge-connectivity).
- Non-Unique with Equal Weights:** Multiple MSTs may exist; all are valid—define tie-breaking if determinism matters.
- Real-World Constraints:** Capacity limits, latency, geography, permissions, and resilience often trump pure cost-minimization.
- Parallel & Scalable:** Kruskal parallelizes (sorting + components); use parallel sort, concurrent union-find.
- Data Freshness Matters:** Graph snapshot/version determines output—changes can alter the MST.
- Business Semantics:** Edge weights must reflect true cost/benefit (e.g., CAPEX/OPEX, risk). Normalize units before optimization.

2 CORE IDEA

- A **spanning tree** has exactly $V-1$ edges and connects all vertices with no cycles.
- An MST minimizes total edge cost, not source-to-target path costs.

CUT PROPERTY (Safe Edge)

For any cut that partitions the vertices into two non-empty sets, the minimum-weight edge crossing the cut is safe — it belongs to some MST.



CYCLE PROPERTY (Discard Edge)

For any cycle, the maximum-weight edge on the cycle cannot be in any MST.



5 TRADE-OFFS

- Optimizes global connection cost, not path latency. An MST can include long edges if they reduce total cost.
- Dynamic updates (edge/vertex changes) are non-trivial.
- Assumes all edge costs are comparable and additive.

When NOT to use MST

- X** Directed graphs needing root-reachable structure → use Minimum Arborescence (Chu-Liu/Edmonds)
- X** Need fastest route between two nodes → use Shortest Path (Dijkstra/Bellman-Ford)
- X** Degree constraints, capacity, or reliability targets → need specialized models (e.g., k -edge-connected)
- X** Need to connect a subset with optional Steiner nodes → use Steiner Tree variants (NP-hard)

8 PRODUCTION

	Fiber/Electrical/Network Planning Lay the cheapest backbone to connect all sites.
	Geographic Pipelines & Roads Minimize construction cost across locations.
	Broadcast / Communication Backbones Connect transmitters/relays cost-effectively.
	Hierarchical Clustering MST-based approaches (single-link).
	Image Segmentation Graph-based segmentation via MST (Felzenszwalb).
	Approximate Layout / PCB / VLSI Wire length minimization heuristics.
	Infrastructure Cost Planning Estimate minimal connection cost of assets.

⚠️ Caveat: MST is fragile—resilience requires extra edges.

11 MENTAL HOOK

An MST connects everything as cheaply as possible, not as quickly as possible.

3 INTERNALS

- Kruskal:** sort edges by weight; add if no cycle (Union-Find).
- Prim:** start anywhere; grow tree by repeatedly adding the minimum-weight edge crossing the cut (min-heap).
- If graph is disconnected, algorithms return a minimum spanning forest (one MST per component).
- Equal weights can yield multiple valid MSTs.
- Assumes undirected, weighted graph with comparable edge costs.

Kruskal (Edge Order)

Sort edges by weight:
BD(1), BC(2), BE(2), DE(2), AD(3), EF(3), AB(4), CF(4), BF(5)

Scan in order → add if no cycle (using Union-Find).

Prim (Growing Tree)

Start at A. At each step, pick minimum crossing edge from tree to outside (using Min-Heap).

Repeat until all vertices are included.

6 DISCUSSION

- Q: MST vs Shortest Path?** A: MST minimizes total cost to connect all nodes; shortest path minimizes cost between two specific nodes.
- Q: Is MST unique?** A: Not always. Equal-weight choices can produce multiple valid MSTs.
- Q: Can edges have negative weights?** A: Yes. MST is still well-defined as long as no negative cycles are required (MST has none).
- Q: Cut property proof idea?** A: Safe edge argument—if a cheaper crossing edge existed, MST with that edge would be cheaper (contradiction).
- Q: How does Kruskal prevent cycles?** A: Only add an edge if its endpoints are in different components (Union-Find).
- Q: Prim with heap—what about stale entries?** A: Use visited/in-tree array to ignore outdated heap entries (lazy deletion).
- Q: Disconnected graph?** A: Returns a minimum spanning forest (MST per component).
- Q: Second-best MST?** A: Replace each MST edge with the min non-tree edge that creates a cycle; pick best increase.
- Q: Dynamic graph updates?** A: Recompute/maintain with dynamic MST data structures (complex), or rebuild when practical.

9 INTERVIEW THINKING

- Verify problem: undirected? connected? weights?
- Clarify objective: minimize total edge sum, connect all vertices.
- Estimate graph density (E vs V^2): sparse or dense?
- Choose approach: Kruskal (DSU) or Prim (heap/matrix).
- Use safe-edge invariant: cut property (add light edge).
- Grow solution until $V-1$ edges (or all components done).
- Handle disconnected graphs → minimum spanning forest.
- State complexity in terms of V and E ; justify choice.
- Consider implementation details: data structures, duplicates, stale heap entries, tie-breaking.
- Test edge cases: single node, equal weights, parallel edges, disconnected, negative weights, large inputs.

KRUSKAL vs PRIM — TRACE (Example Graph Above)

Kruskal Trace					Prim (Start at A) Trace				
Step	Edge	Weight	Action	UF Sets After	Step	Edge Added	Weight	Total Cost	In-Tree Set
1	BD	1	Add	{A,B,C,F},{D} → {A,B,C,D,E,F}	1	A-D	3	3	{A,D}
2	BC	2	Add	{A,B,C,D,E,F}	2	D-B	1	4	{A,B,D}
3	BE	2	Skip (cycle)	{A,B,C,D,E,F}	3	B-C	2	6	{A,B,C,D}
4	DE	2	Add	{A,B,C,D,E,F}	4	D-E	2	8	{A,B,C,D,E}
5	AD	3	Add	{A,B,C,D,E,F}	5	E-F	3	10	{A,B,C,D,E,F}
6	EF	3	Add	{A,B,C,D,E,F}					
MST Edges: {BD, BC, DE, AD, EF} Total Cost = 10					MST Edges: {AD, DB, BE, DE, EF} Total Cost = 10				

DYNAMIC PROGRAMMING

Senior Software Engineer / Technical Lead Interview Prep (2–3 Minute Review)

1. WHY

Eliminate repeated subproblems in optimization, counting, or feasibility problems by solving each subproblem once and reusing the result.

Trade memory for time.

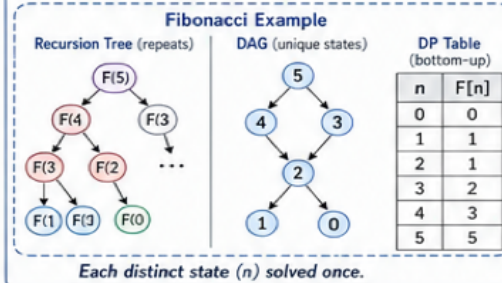
Good for problems with:

- ✓ Optimal substructure
- ✓ Overlapping subproblems

Goal: Turn exponential $\rightarrow$ polynomial (or pseudo-polynomial) time.

2. CORE IDEA

- 1 **Define state:** what summarizes the subproblem.
- 2 **Recurrence / transition:** how to build from smaller states.
- 3 **Base cases:** smallest / trivial states.
- 4 **Evaluation order:** any order that respects dependencies.
- 5 **Answer:** value of target state.



3. INTERNALS

- **Top-down (memoization):** DFS + cache (hash map / array). Stores computed states, returns on hit.
- **Bottom-up (tabulation):** iterative fill of table in dependency order.
- **State key encoding:** indices, bitmasks, tuples, compressed representation. Keep minimal.
- **Dependency DAG:** directed edges from a state to the states it depends on.
- **Sentinel / uncomputed:** e.g., -1, null, INF, NaN to mark "not computed".
- **Space optimization:** rolling arrays, 1D compression, bitset tricks, divide & conquer DP.
- **Reconstruction:** store parent/choice/pointer or separate decision table to rebuild the solution.
- **Numeric concerns:** overflow (use 64-bit / BigInt / mod), INF carefully (avoid $\text{INF} + x$ overflow).

4. COMPLEXITY

Time $\approx$ (number of states) $\times$ (transitions per state)
Space $\approx$ states (+ reconstruction data)

Problem	State Definition	Time	Space
Fibonacci	n	O(n)	O(n) / O(1)*
Grid Paths (rxc)	i, j	O(rc)	O(rc) / O(c)*
0/1 Knapsack	i, w	O(nW)	O(nW) / O(W)*
LCS (len. n, m)	i, j	O(nm)	O(nm) / O(m)*
Subset DP	mask	O(2^n)	O(2^n)

* with rolling array / space compression.

Warning: Many DPs are pseudo-polynomial (e.g., O(nW)). Large numeric ranges (W, sum, value) can make them infeasible (e.g., W up to 10^9).

5. TRADE-OFFS

Strengths

- ✓ Systematic, predictable performance.
- ✓ Turns exponential recursion into polynomial.
- ✓ Often simpler and more robust than clever greedy.

Costs / Risks

- ✗ State explosion $\rightarrow$ time/space blow-up.
- ✗ Extra memory for tables / caches.
- ✗ Recurrence or order mistakes are subtle.

When NOT to use DP

- No overlapping subproblems.
- Greedy choice property applies (greedy is optimal).
- A graph algorithm (Dijkstra, max-flow, etc.) fits better.
- Constraints make state space too large.
- Approximation / heuristics are required by problem.

6. DISCUSSION (Interview Qs)

- 1 How do you recognize a problem is DP-able?
- 2 Would you use memoization or tabulation? Why?
- 3 How do you define the minimal state?
- 4 What are the transitions / recurrence?
- 5 What is the evaluation order and why is it valid?
- 6 How do you handle base cases and boundaries?
- 7 How do you reconstruct the actual solution?
- 8 Can you compress space safely? How?
- 9 How do digit DP, tree DP, or bitmask DP work?
- 10 How do you handle negative cycles (not allowed in shortest-path DP)?
- 11 Is the DP pseudo-polynomial? What are limits?

7. RELATED TOPICS / PATTERNS

- 📂 Recursion & Memoization
- 🔗 DAG Shortest Path (topological DP)
- 👑 Greedy Algorithms
- 🔍 Backtracking (for state-space exploration)
- 🔄 Divide & Conquer DP (e.g., DP on intervals)
- 📊 Bellman Equations (expectation, value iteration)
- 🌲 Dynamic Programming on Trees (Tree DP)
- 🔄 Finite State Machines / Automata DP

8. WHERE IT SHOWS UP (Production)

- 🧬 Sequence Alignment (Bioinformatics)
- 💰 Resource Allocation & Budgeting
- 📦 Pricing & Packing (Knapsack, Cuts)
- 📄 Diff / Edit Distance (Text, Code)
- 📅 Scheduling (Job Shop, Intervals)
- 📍 Routing with Constraints
- 🔗 Parser / Viterbi (NLP, HMMs)
- 📊 Inventory Planning & Demand Forecast
- 🧠 ML Decoding (Beam / DP)
- 🗄️ Database Query Optimization

9. INTERVIEW THINKING FLOW

- 1 Start with brute force / naive recursion.
- 2 Identify repeated subproblems (overlap).
- 3 Define the minimal state that captures subproblem.
- 4 Write the recurrence / transition.
- 5 Establish base cases.
- 6 Determine valid evaluation order.
- 7 Analyze complexity: states $\times$ transitions.
- 8 Optimize space if needed (rolling / compression).
- 9 Reconstruct solution if required.
- 10 Test on tiny, impossible, and extreme cases.

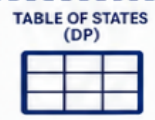
10. SENIOR ENGINEER NOTES

- ★ State definition is the real design. A good state makes the rest easy.
- ★ Dimensions = complexity. Extra dimensions multiply cost.
- ★ Space compression may destroy reconstruction. Plan ahead.
- ★ Memoization computes only reachable states. Great for sparse state spaces.
- ★ Pseudo-polynomial can still be infeasible. Check value ranges early.
- ★ Validate recurrence with brute force on tiny inputs.
- ★ Think in invariants and boundaries. Off-by-one errors are common.
- ★ Consider cache locality and constant factors for large tables.
- ★ Document state meaning and transitions clearly for maintainability.

11. MENTAL HOOK



Dynamic programming turns a recursion tree into a table of states solved once.



GREEDY ALGORITHMS

SENIOR SOFTWARE ENGINEER / TECHNICAL LEAD INTERVIEW CHEATSHEET

1

WHY

Make locally best, irreversible choices to achieve global optimum with simple, fast algorithms when structure permits.

- ✓ Local decision made from current state
- ✓ Commit (irreversible), never reconsider
- ✓ Reduce problem to a smaller instance
- ✓ Repeat until complete
- ✓ Correctness depends on problem structure (not all problems)

★ Goal: optimal solution with low time, low space, and easy implementation.

2

CORE IDEA

Choose the best available option now, commit to it, and reduce the problem.

Requires GREEDY-CHOICE PROPERTY + OPTIMAL SUBSTRUCTURE

Example: Interval Scheduling (select max non-overlapping intervals by earliest finish)

Step	Chosen Interval	Remaining Intervals	Finish
0	— (start)	[1,3] [2,5] [4,6] [6,7] [5,8]	—
1	↓ [1, 3]	[4,6] [6,7] [5,8]	3
2	↓ [4, 6]	[6,7] [5,8]	6
3	↓ [6, 7]	[5,8]	7
4	↓ Done	[]	—

Result: {[1,3], [4,6], [6,7]} (size 3) — optimal

3

INTERNALS

- **Selection mechanism:** sorting or priority queue often drives the next choice.
- **Feasibility invariant:** maintain a property that keeps partial solution extendable.
- **Correctness tools:**
 - **Exchange Argument:** swap choices to show an optimal solution can be transformed to match greedy.
 - **Stays-Ahead (Induction):** greedy solution is never worse at any prefix.
 - **Cut Property (Graphs):** greedy picks a safe edge crossing the cut.
- **Matroid intuition:** greedily optimal on matroids (e.g., MST, activity selection).
- **Tie-breaking & determinism:** define stable rules to ensure reproducible results.

4

COMPLEXITY



Typical Pattern

Sort by key $O(n \log n)$ + Single pass / scan $O(n)$
Total: $O(n \log n)$



With Heap / Priority Queue

Push/Pop per element $O(\log n) \rightarrow O(n \log n)$ total



Graph Greedy Examples

- **Kruskal (MST):** sort edges $O(E \log E) \sim O(E \log V)$ + Union-Find $O(\alpha(V))$ per op
- **Prim (MST):** with binary heap $O(E \log V)$ or $O(E + V \log V)$
- **Dijkstra (nonnegative weights):** $O((V + E) \log V)$ with binary heap



Space

$O(n)$ to $O(m)$ depending on structure (arrays, heap, DSU, adjacency lists)

Greedy does NOT imply $O(n)$.
Sorting/heap often dominate time.

5

TRADE-OFFS

STRENGTHS

- ✓ Simple to implement and reason about (once proven)
- ✓ Fast: usually $O(n \log n)$ or $O(n \log n)$
- ✓ Streamable / online in many variants
- ✓ Low memory footprint

WEAKNESSES

- ✗ Correctness is highly problem-specific
- ✗ Irreversible choices can be globally bad
- ✗ Proof is required (intuition is not enough)

WHEN NOT TO USE (Counterexamples)

- 0/1 Knapsack: greedy by value/weight fails
 - Dijkstra with negative edge weights
 - Problems with strong future interactions (e.g., knapsack, set cover exact, TSP)
- Use DP / Backtracking / Branch & Bound instead.

6

DISCUSSION (Talking Points)

- ? How would you prove this greedy algorithm is correct?
- ? Which property allows greedy choice here?
- ? Can you give an exchange argument?
- ? Can you construct a counterexample where greedy fails?
- ? How does fractional knapsack differ from 0/1?
- ? Why does earliest finish time work for interval scheduling?
- ? Why is Huffman coding greedy optimal?
- ? Why is Dijkstra's greedy choice safe only with nonnegative edges?
- ? How do you handle ties? Does it affect correctness or only determinism?
- ? Can we get approximation guarantees with greedy? What's the bound?

7

RELATED



Dynamic Programming — solves overlapping subproblems when greedy fails.



Matroids — class of problems where greedy is provably optimal.



Priority Queues / Heaps — efficient selection of next best candidate.



Sorting — establishes global order to enable greedy scanning.



Minimum Spanning Tree — Cut Property justifies Kruskal/Prim choices.



Dijkstra's Algorithm — greedy by smallest distance (nonnegative weights).



Interval Scheduling — earliest finish greedy is optimal.



Huffman Coding — greedy merging of least frequencies yields optimal prefix code.

8

PRODUCTION USES



Scheduling & Allocations — meeting rooms, CPU jobs, airline gates, crew rostering (heuristics).



Compression — Huffman coding, optimal prefix codes.



Network Spanning — MST for wiring, clustering, network design.



Routing (Nonnegative) — shortest paths, navigation, latency-aware routing.



Caching (Heuristics) — LRU/LFU/GreedyDual-Size (note: no perfect offline greedy).



Resource Packing (Approximations) — on-line bin packing, subset sum approximations.



Ad Allocation / Ranking — greedy by CTR or ROI under budget/constraints (heuristics).



Load Shedding / Throttling — drop/skip least valuable work to protect SLOs.

Great when near-optimal fast decisions are needed and proof or guarantees are known.

9

INTERVIEW THINKING PLAYBOOK

- 1 Enumerate choices and constraints clearly.
- 2 Propose a greedy rule (what is "best" now?).
- 3 State invariant / feasibility condition.
- 4 Try to break it: search for counterexample.
- 5 Prove correctness (exchange / stays-ahead / cut).
- 6 Design data structures (sort / heap / DSU / PQ).
- 7 Write clean implementation (edge cases!).
- 8 Analyze complexity (time & space).
- 9 Tie handling, duplicates, boundaries, overflow.
- 10 Compare with DP/backtracking solutions.

Communicate trade-offs and assumptions.

10

SENIOR NOTES

- ★ Greedy requires a proof, not intuition. Never assume it works.
- ★ Understand how the objective and constraints make greedy choices "safe".
- ★ Tie-breaking policy can affect determinism and downstream systems.
- ★ Approximation? State the guarantee (ratio) and its conditions.
- ★ Online greedy (no foresight) can be very different from offline greedy with sorted info.
- ★ Measure business impact and operational trade-offs when using heuristic greedy in production.

💡 Be precise about assumptions and failure modes.

11

MENTAL HOOK

“ Greedy is correct only when the future cannot punish the best safe choice now. ”

PROOF TOOLKIT AT A GLANCE

Tool	When to Use	Idea
Exchange Argument	Show there exists an optimal solution that makes the same choice as greedy	Swap choices without harming optimality until it matches greedy
Stays-Ahead (Induction)	Compare greedy and optimal solutions step by step	Greedy never lags behind at any prefix
Cut Property (Graphs)	MST problems	Minimum weight edge crossing any cut is safe
Matroid Structure	Feasible sets closed under hereditary + exchange	Greedy by weight gives optimal independent set

A proof > a counterexample against doubt.

BACKTRACKING

Senior Software Engineer / Technical Lead Interview Prep – 2–3 Minute Review

1 WHY

Systematically explore combinatorial choices while pruning partial solutions that cannot succeed.

- Handles constraints, enumeration, puzzles.
- Explores a state-space tree of choices.
- Pruning avoids wasting time on dead ends.
- Finds one or all valid solutions (or optimal with branch & bound).

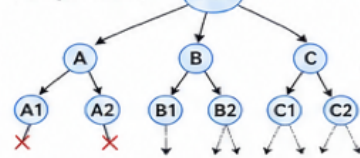
Great for:

- ✓ Constraint satisfaction (e.g., Sudoku, N-Queens)
- ✓ Enumerating permutations / combinations
- ✓ Packing, scheduling, routing with constraints
- ✓ Search where constraints make most branches fail

2 CORE IDEA

choose → apply → recurse → undo

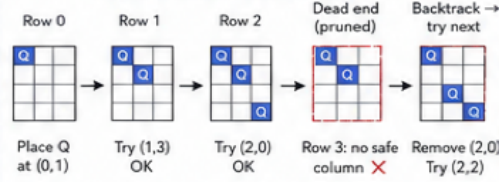
State-space tree (example domain)



- Feasibility predicate prunes.
- Goal test records or returns solution.

○ partial state ✓ goal (solution) ✗ pruned (infeasible)

Tiny N-Queens trace (N = 4)



4 COMPLEXITY

- Worst case exponential / factorial.
- Branching factor b , depth d .

Problem Type	Typical Count	Worst-Case Time
Permutations of n	$n!$	$O(n!)$
Subsets of n	2^n	$O(2^n)$
General backtracking	$\approx b^d$	$O(b^d)$

- Space: $O(d)$ for recursion + $O(|\text{state}|)$.
- Output-sensitive: time is $\Omega(\#\text{solutions})$.
- Pruning, ordering, propagation change practical time, NOT worst-case bound.

Example: N-Queens ($N \times N$)

- Search space (row by row): N^n
- Strong pruning reduces explored nodes dramatically.
- Time $\sim O(c^N)$ with $c < N$ (empirically $\sim O((1.2)^N)$)

5 TRADE-OFFS

Pros

- ✓ Complete (when enumeration is finite)
- ✓ Expressive for complex constraints
- ✓ Easy to add/remove constraints
- ✓ Finds all solutions or first feasible/optimal

Cons

- ✗ Can explode combinatorially
- ✗ Deep recursion → stack / time
- ✗ Mutation bugs → incorrect results
- ✗ Hard to design good pruning/order

When NOT to use

- Polynomial-time algorithm exists (graph/DP/greedy).
- Huge unstructured domain with weak constraints.
- Approximation / heuristic is sufficient.
- Hard real-time guarantees with unknown worst-case.

3 INTERNALS

- 1 Mutable state + exact undo: Apply choice, recurse, then undo to restore.
- 2 Path / used sets: Track current partial solution & used items.
- 3 Candidate ordering: Good order → earlier pruning, faster wins.
- 4 Recursion stack: Implicit stack holds choices and state.
- 5 Copy vs mutate: Copy simplifies; mutate faster, needs undo.
- 6 Constraint propagation: After each choice, update constraints to prune more.
- 7 Branch and bound: Add objective bound to prune non-promising.
- 8 Memoization: Cache states only when future depends solely on encoded state.
- 9 Early exit / all solutions: Stop at first goal or collect all.

Skeleton (pseudo)

```
def backtrack(state):
    if is_goal(state):
        record(state); return True/False
    for choice in order(candidates(state)):
        if not feasible(state, choice): continue
        apply(state, choice)
        if backtrack(state): return True # or collect
        undo(state, choice)
    return False
```

6 DISCUSSION

- 1 How is backtracking different from DFS?
- 2 How do you ensure pruning is correct (sound)?
- 3 How do you handle duplicates in permutations?
- 4 What is the time complexity of N-Queens?
- 5 How does constraint propagation help (e.g., Sudoku)?
- 6 How does branch-and-bound extend backtracking?
- 7 When is memoization useful in backtracking?
- 8 How would you make it iterative (explicit stack)?
- 9 How would you support cancellation / timeouts?
- 10 How would you parallelize the search?

7 RELATED

DFS (Depth-First Search)	Traversal strategy
Recursion	Call stack management
Constraint Satisfaction (CSP)	Variables + constraints
Branch and Bound	Optimization with bounds
DP / Memoization	Reuse overlapping states
SAT Solving	Boolean CSP at scale
nPr / nCr	Permutations / Combinations
Game Search (Minimax / Alpha-Beta)	Pruned game trees

8 PRODUCTION USES

Configuration solving	Feature flags, system configs
Scheduling	Shifts, resource allocation
Dependency / version resolution	Build tools, package managers
Test data generation	Combinations under constraints
Regex / parser paths (caution)	Exploring alternatives; beware blow-up
SAT	Theorem proving / SAT / SMT
Routing with constraints	Capacities, time windows, policies
Access-policy combinations	RBAC / ABAC policy evaluation
Puzzle & game AI	Sudoku, crosswords, chess, puzzles

9 INTERVIEW THINKING

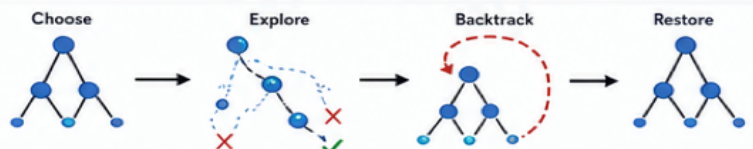
- 1 Define the state and what a solution looks like.
- 2 Identify choices at each step (candidates).
- 3 Define goal test (complete solution?).
- 4 Define validity & pruning (necessary conditions).
- 5 Choose ordering/heuristics to prune early.
- 6 Design apply() and undo() — maintain invariants.
- 7 Handle duplicates (sets, counts, ordering rules).
- 8 Ensure termination (finite choices, progress).
- 9 Analyze complexity: estimate b and d .
- 10 Decide: first solution, all solutions, or optimal? Plan early stop / enumeration.

Test with edge cases: empty, single item, all invalid, max size, duplicates, tight constraints.

10 SENIOR NOTES

- ✓ Undo symmetry is correctness-critical: every apply must have exact inverse.
- ✓ Pruning must be necessary-condition safe. Never prune a potentially valid branch.
- ✓ Candidate ordering can dominate runtime (MRV, degree heuristic, fail-first).
- ✓ Cap & cancel: guardrails for adversarial inputs; expose time/solution limits.
- ✓ Copy vs mutate: copying state simplifies correctness but increases memory/time.
- ✓ Memoize only when future depends solely on encoded state (Markov property).
- ✓ Instrumentation matters: counts, depth histograms, prune reasons, hot spots.
- ✓ Design for parallel search: split at top levels; use work-stealing; dedupe results.
- ✓ Robustness: validate inputs, handle unsatisfiable early, return partials if useful.

11 MENTAL HOOK



Backtracking explores a choice, proves it hopeless or complete, then restores the world.

DIVIDE & CONQUER

Senior Software Engineer / Technical Lead Interview Prep – 2-3 Minute Review

1 WHY

- Solve a large problem by breaking it into independent smaller subproblems, solve them (often recursively), then combine.
- Exposes parallelism across subproblems.
- Improves cache locality via smaller working sets and locality-friendly layout.
- Scales naturally with input size.

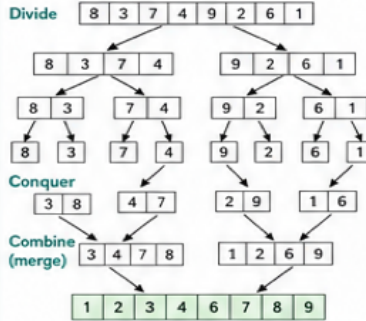


Goal: Reduce complexity per subproblem while keeping total work near-optimal.

2 CORE IDEA

1. **Divide**: partition into smaller subproblems.
2. **Conquer**: solve each recursively.
3. **Combine**: merge or aggregate results.
4. **Base case**: stop when problem is small enough to solve directly.

Example: Merge Sort



Not the same as:

Decrease-and-Conquer
(reduce to one smaller problem)

Example: Binary Search looks at half but does not solve both halves.



Dynamic Programming
(overlapping subproblems)

Example: Fibonacci recomputes same states without memoization.



3 INTERNALS

- **Recursion frames**: each call has local variables, return address, parameters.
- **Partitioning cost $f(n)$** : time to split and prepare subproblems.
- **Combine cost**: time and memory to merge/aggregate results; may require temporary buffers.
- **Balance matters**: balanced splits give smaller depth; unbalanced $\rightarrow$ worse cases.
- **Cutoff threshold**: stop recursion at size $\leq k$ and use iterative/base algorithm (e.g., insertion sort).
- **Parallel model**: fork-join tasks for subproblems; join at combine.
- **Cache-oblivious layouts**: recursive divide naturally improves locality.
- **Stack depth**: for balanced divide by b , depth $\approx \log_b n$; watch stack limits.

4 COMPLEXITY

General recurrence:

$$T(n) = a \cdot T(n/b) + f(n)$$

$a \geq 1$ subproblems, each of size n/b , plus $f(n)$ work to divide/combine.

Master Theorem (regularity holds)

Case	Condition	$T(n)$
1	$f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$	$\Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a} \log^k n)$ for $k \geq 0$	$\Theta(n^{\log_b a} \log^{k+1} n)$
3	$f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$ and $a \cdot f(n/b) \leq c \cdot f(n)$ for some $c < 1$	$\Theta(f(n))$

Common Examples

Algorithm	a	b	$f(n)$	$T(n)$ (Time)	Space (extra)
Merge Sort	2	2	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n)$
Binary Search	1	2	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Quicksort (avg)	2	2	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(\log n)$
Quicksort (worst)	n	n	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n)$
Strassen (matrix mult.)	7	2	$\Theta(n^2)$	$\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$	$\Theta(n^3)$
FFT (Cooley-Tukey)	2	2	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n)$

Recursion space = stack $O(\log_b n)$ (balanced) + temp buffers for combine (often $O(n)$).

5 TRADE-OFFS

Pros

- ✓ Modular and easier to reason about.
- ✓ Natural parallelism (embarrassingly parallel subproblems).
- ✓ Good cache behavior with recursive layouts.
- ✓ Scales with data size; often near-optimal work.

Cons / Costs

- Recursion overhead (calls, stack).
- Extra memory for buffers / temporary storage.
- Imbalanced splits lead to poor worst-case.
- Combine step can dominate (e.g., large merges).

When NOT to Use

- ⊗ Subproblems overlap heavily $\rightarrow$ use DP / memoization.
- ⊗ Problem cannot be split independently.
- ⊗ Data too small $\rightarrow$ overhead outweighs benefit.
- ⊗ Synchronization/communication cost in parallel exceeds useful work.
- ⊗ Highly skewed or adversarial inputs without mitigation.



Rule of Thumb: Use when split is easy, work is balanced, and combine is linear (or cheaper) than solving the whole.

6 DISCUSSION

1. How do you apply the Master Theorem to analyze a given recurrence?
2. Why is balanced splitting important?
3. What causes quicksort's worst case and how do you avoid it?
4. Merge sort uses $O(n)$ extra memory—how would you reduce it or avoid it?
5. How do you choose a parallel cutoff threshold?
6. When would you prefer divide & conquer over DP?
7. How does tail recursion (or iteration) help here?
8. Which sorting algorithms are stable? Why might stability matter?
9. How would you design divide & conquer for a custom domain problem?
10. What metrics would you measure to validate performance (work vs critical path, memory, cache misses)?

7 RELATED

- **Recursion fundamentals** (base case, stack, recursion trees).
- **Sorting**: merge sort, quicksort, heap sort.
- **Searching**: binary search.
- **FFT (Fast Fourier Transform)**.
- **Parallelism**: fork-join model, work stealing.
- **Cache-oblivious algorithms** and layouts.
- **Computational geometry**: closest pair of points.
- **Dynamic Programming** (contrast: overlapping subproblems + memoization/tabulation).

8 PRODUCTION USES

- **Parallel sort** of big data in-memory and external sorts.
- **Image/tile processing**: pyramids, mipmaps, quadrees.
- **Map-Reduce** style partitions of datasets.
- **Spatial indexes**: quadrees, k-d trees, R-trees.
- **Compiler optimization** passes over AST/IR (divide function, basic blocks, dominator trees).
- **Large-file processing**: divide files into chunks, process in parallel, then combine.
- **Numerical algorithms**: FFTs, n-body (Barnes-Hut), matrix mult. (Strassen), big integer multiplication.



Caveat: In distributed systems, data skew can break balance—plan for rebalancing and straggler mitigation.

9 INTERVIEW THINKING PATH

1. Identify the problem and possible way(s) to split it.
2. Check independence of subproblems (no overlap?).
3. Define base case and cutoff threshold k .
4. Decide how to solve subproblems (recursively/iteratively).
5. Plan the combine step (associative? stable? memory?).
6. Write recurrence $T(n) = aT(n/b) + f(n)$.
7. Analyze complexity; check Master Theorem conditions.
8. Consider balance: best/avg/worst cases and invariants.
9. Evaluate memory: stack depth + temporary space.
10. Consider parallelization and synchronization costs.
11. Test edge cases: skewed data, tiny inputs, duplicates, already-sorted, adversarial.



Always compare alternatives (DP, greedy, iterative, randomization) and pick simplest that works.

10 SENIOR NOTES

- ★ Partition quality is a performance invariant—guard it.
- ★ Total work and critical path (span) both matter.
- ★ Parallel speedup needs coarse enough tasks; too fine $\rightarrow$ overhead dominates.
- ★ Memory traffic (bandwidth) often dominates; design combine to be linear and streaming.
- ★ Randomized strategies (e.g., pivot choice) manage adversaries and stabilize performance.
- ★ Measure in production-like environments; microbenchmarks can mislead.
- ★ Master Theorem is powerful but not universal—know other tools (Akra-Bazzi, recursion tree, substitution).

11 MENTAL HOOK



Divide and conquer wins when splitting creates independent work cheaper to combine than to solve whole.

QUICK REFERENCE: RECURSION TREE EXAMPLES

